

ASSIGNMENT 6

GUI-Based Multi-Client Chat Application Using TCP

Roll No: 5130108

Name: Tanvi Chaudhari

Tezpur University | ISEA Phase 3 Internship

1. Objective

The objective of this assignment was to convert the terminal-based TCP chat application built in Assignment 5 into a graphical desktop application. The existing server and socket communication logic were reused with minimal modification, and a Tkinter-based GUI client (client_gui.py) was built around it. This introduced GUI programming, event-driven programming, and multithreading to keep the interface responsive while messages are received in the background.

2. GUI Design

The client has two windows presented in sequence:

- **Login window** - fields for Server IP, Port, Username, and an optional Password, with a Connect button and input validation (empty usernames are rejected and connection errors are shown inline).
- **Chat window** - opened only after a successful login. It contains a scrollable, auto-scrolling message area, an Online Users list, a message entry box, a 'Private to selected user' checkbox, and Send / Disconnect buttons, plus a status label showing the connection state.

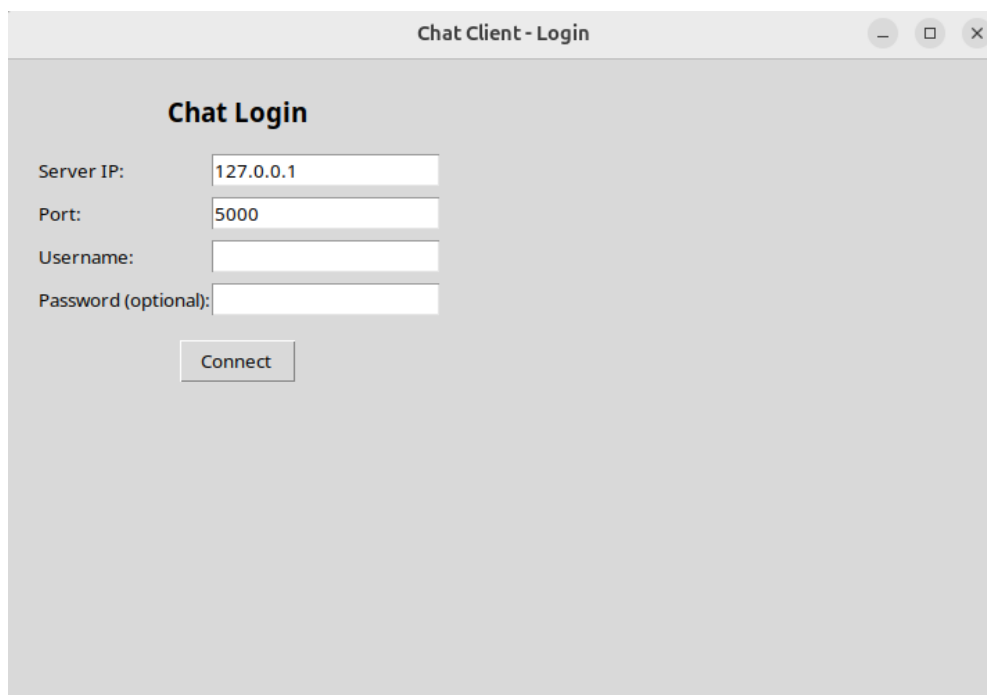


Fig 2.1 - GUI Login Window

3. System Architecture

The application follows a client-server architecture over TCP sockets. The server listens on port 5000 and spawns one thread per connected client. Each GUI client opens one TCP connection to the server and runs a dedicated background thread to receive messages, so the main GUI

thread never performs a blocking socket call.

Communication uses newline-delimited JSON messages, keeping the networking layer independent of the GUI. Example message types:

```
{"type": "login", "username": "alice"}  
{"type": "broadcast", "text": "hello everyone"}  
{"type": "private", "to": "bob", "text": "hi bob"}  
{"type": "userlist", "users": ["alice", "bob"]}  
{"type": "disconnect"}
```

Thread / Data Flow

Main GUI thread (Tkinter event loop) → user clicks Send → `send_json()` writes to the socket directly. Incoming data is handled the other way: background `recv_loop()` thread reads from the socket and pushes parsed JSON messages onto a thread-safe queue. `Queue()`; the main thread polls this queue every 100ms via `root.after()` and updates the widgets. This separation avoids any direct cross-thread widget access, which Tkinter does not support safely.

4. Components Reused from Assignment 5

- TCP socket server accepting multiple clients with one thread per connection.
- Username-based login and duplicate-username rejection.
- Broadcast messaging logic (sending a message to all connected clients).
- Private messaging logic (routing a message to a single named recipient).
- Online user list maintenance and join / leave notifications.
- Server-side activity logging to `server.log`.

Only the client side was rewritten: the terminal input/output loop of Assignment 5 was replaced with Tkinter widgets, while the underlying socket send/receive calls and JSON message format were kept unchanged.

5. GUI Design Decisions

- **ttk widgets** were preferred over classic tk widgets for a cleaner, more native look (Entry, Button, Frame, LabelFrame, Checkbutton).
- **ScrolledText** was used for the chat log so long conversations remain readable, with automatic scroll-to-bottom on every new message.
- **Listbox** for online users doubles as the target picker for private messages - selecting a name and ticking 'Private to selected user' avoids the need to type usernames manually.
- **queue.Queue + root.after()** was chosen over directly touching widgets from the network thread, since Tkinter is not thread-safe.

- A single status label gives constant visual feedback on whether the client is connected or disconnected.

6. Testing Results

The application was tested using one server host (h1) and four client hosts (h2-h5) inside a Mininet 'single,5' topology, with connectivity verified using pingall before testing.

6.1 Successful Login and Connection

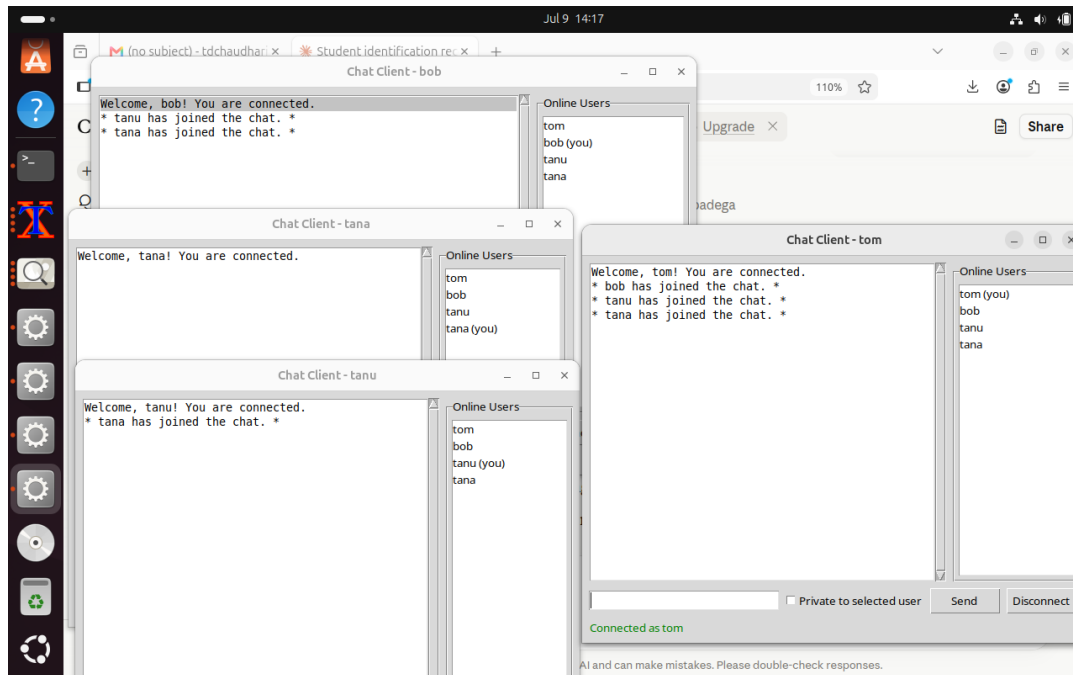


Fig 6.1 - Four clients (tom, bob, tanu, tana) connected simultaneously; the Online Users list updates identically on every client.

6.2 Main Chat Window

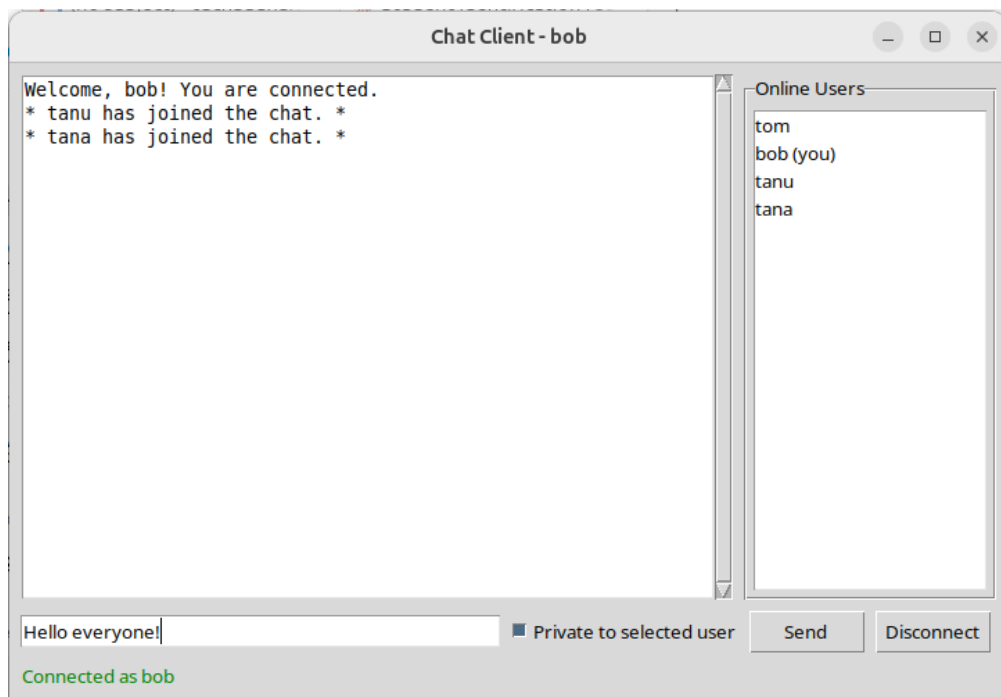


Fig 6.2 - Chat window for client 'bob' showing the message area, online users panel, input box, and Send / Disconnect buttons.

6.3 Broadcast Messaging

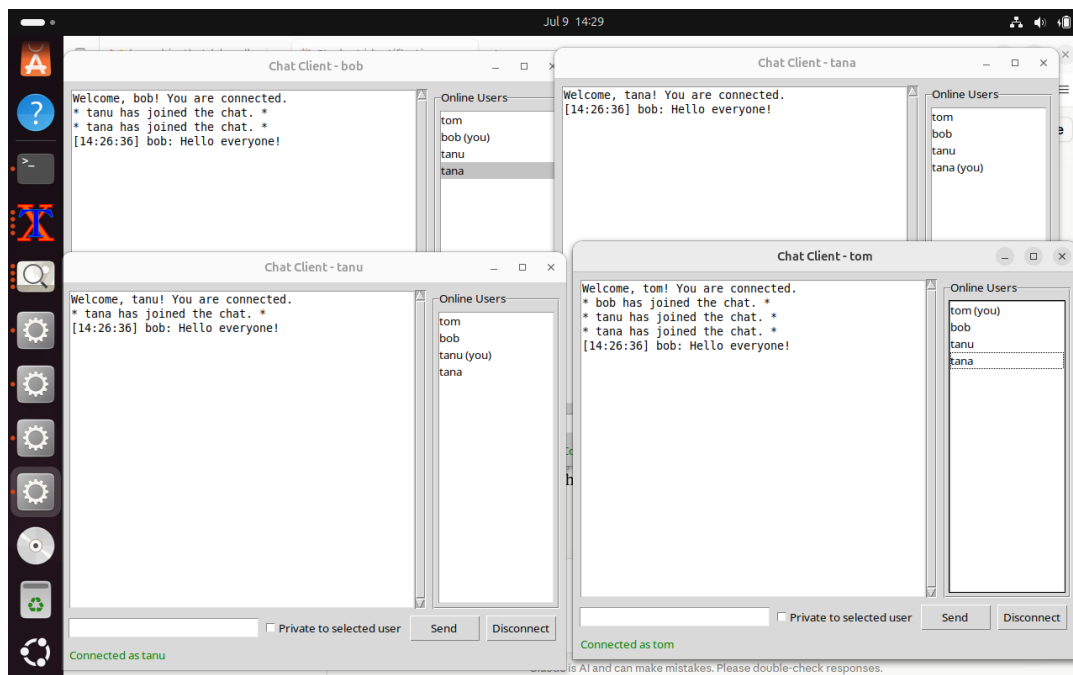


Fig 6.3 - A broadcast message sent by 'bob' is received by all other connected clients (tom, tanu, tana) and by the sender.

6.4 Private Messaging

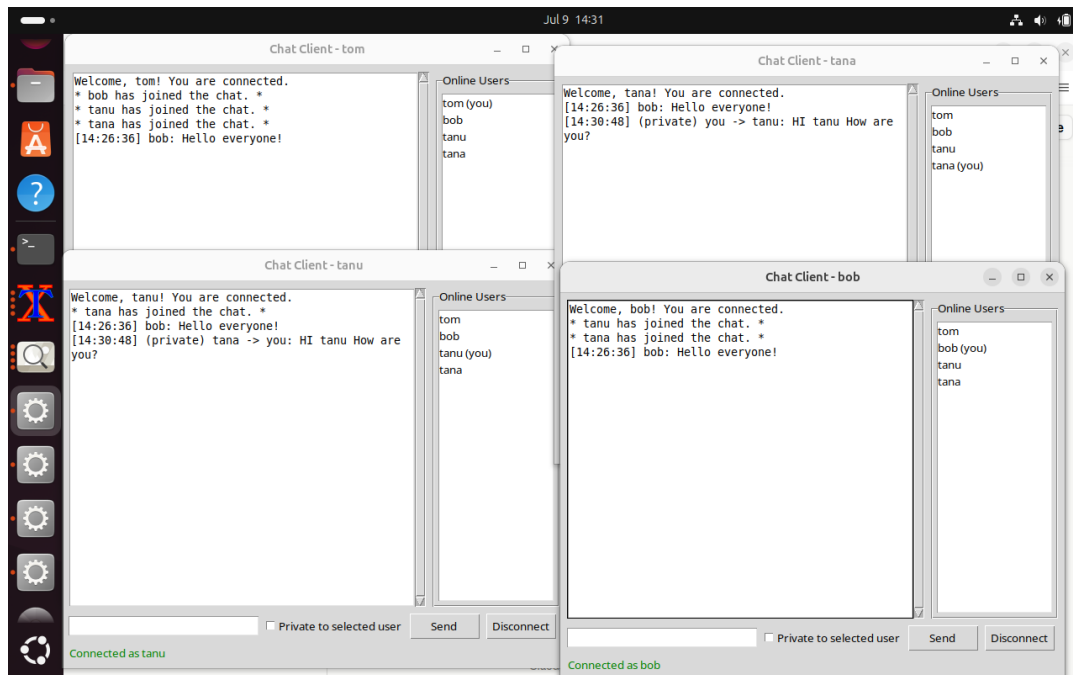


Fig 6.4 - A private message from 'tana' to 'tanu' is visible only to the sender and the selected recipient, not to the other clients.

6.5 User Leaving

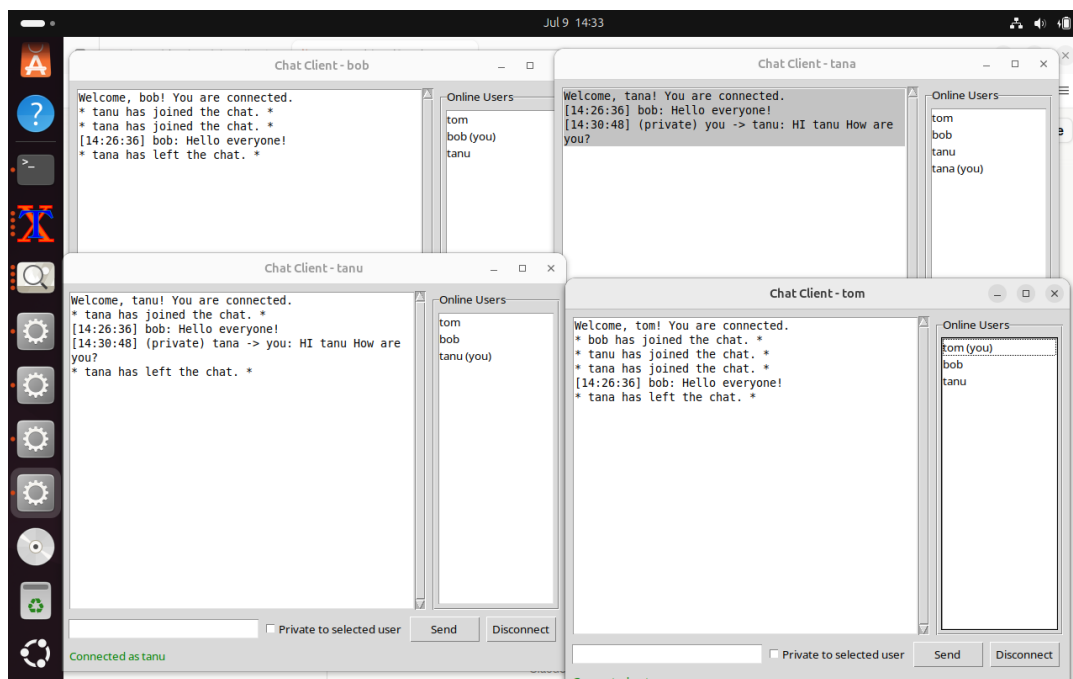


Fig 6.5 - After 'tana' disconnects, a 'tana has left the chat' notice appears on all remaining clients and the Online Users list updates.

6.6 Server Log

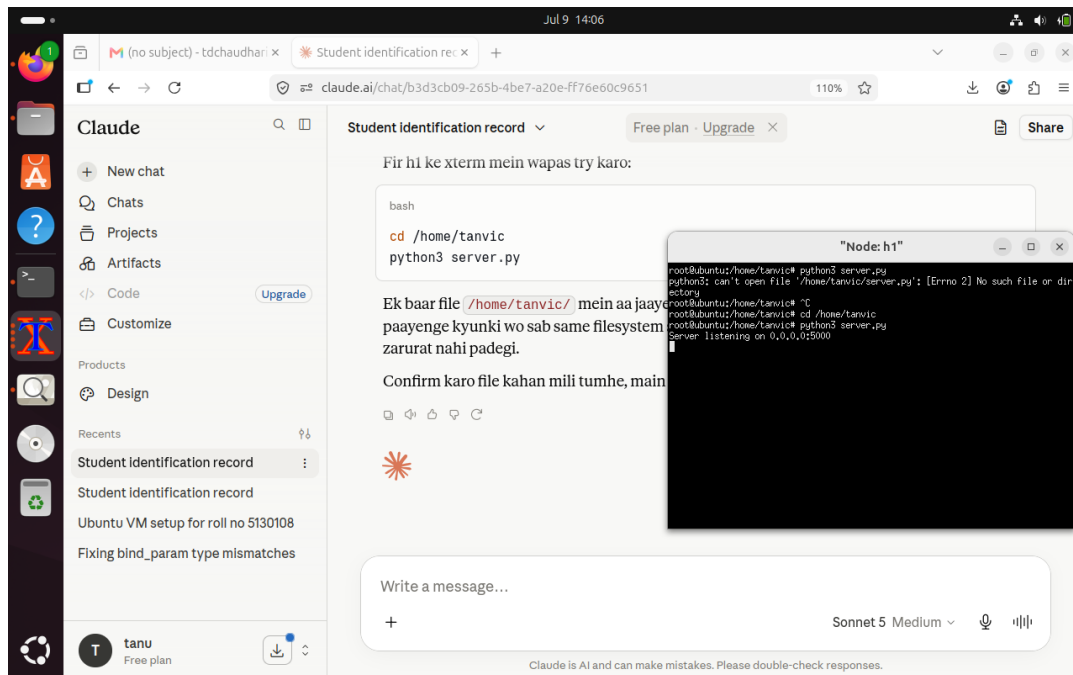


Fig 6.6 - Server (h1) listening on 0.0.0.0:5000 and accepting client connections.

7. Wireshark Verification

Traffic was captured on interface 'any' with the display filter `tcp.port == 5000` applied, isolating the chat application's traffic from the four Mininet client hosts (10.0.0.2 - 10.0.0.5) and the server (10.0.0.1).

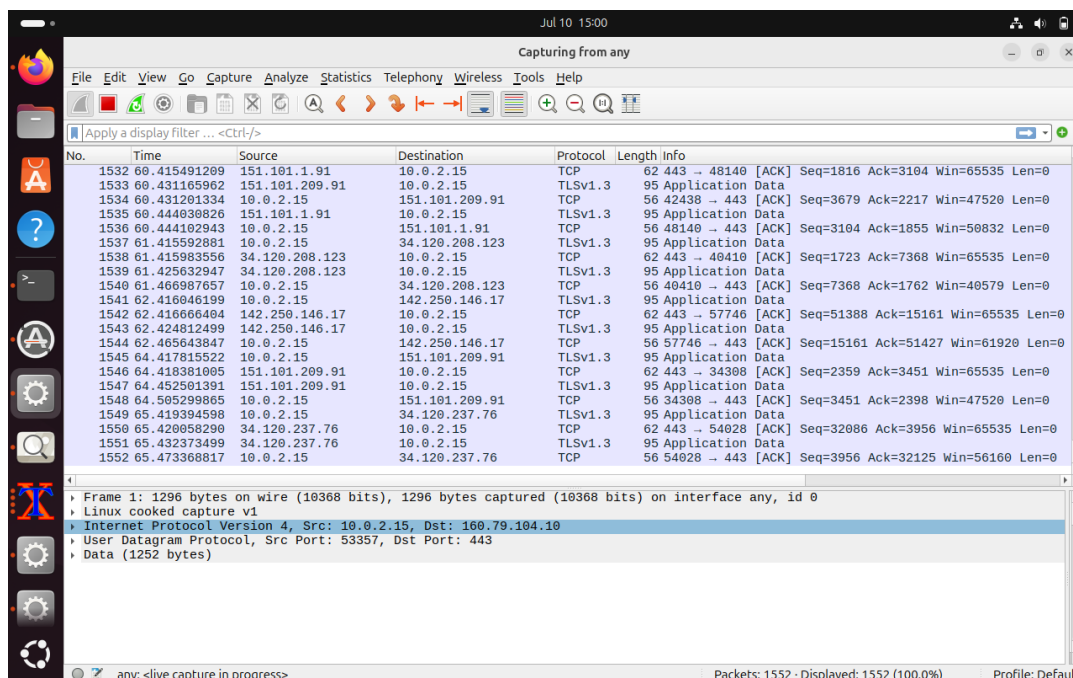


Fig 7.1 - Filtered capture (`tcp.port == 5000`) showing PSH/ACK data packets exchanged between each client and the server. Each packet corresponds to one newline-delimited JSON chat message.

The capture confirms that: (1) each client establishes an independent TCP connection to the server on port 5000; (2) chat messages are transmitted as discrete PSH/ACK data segments; and (3) the server responds to each client individually, matching the one-thread-per-client

design.

8. Reflection Answers

Why should networking logic and GUI code be separated?

Separating the two keeps each concern independent and easier to maintain: the socket/JSON protocol can be tested or reused (e.g., with a different front end) without touching the GUI, and the GUI can be redesigned without risking the communication logic. It also avoids blocking calls inside the event loop - if socket code were mixed directly into button handlers, a slow or stalled network call would freeze the entire interface.

Why is a background thread required?

Tkinter runs a single-threaded event loop responsible for drawing and responding to user input. `socket.recv()` blocks until data arrives, so calling it on the main thread would freeze the window whenever no message is currently available. Running `recv()` in a separate thread lets the GUI keep responding to clicks and typing while messages are received in the background; the two threads communicate safely through a `queue.Queue()`.

Which components from Assignment 5 were reused?

The server implementation, the TCP socket handling, broadcast and private messaging logic, online user list management, join/leave notifications, and server-side logging were all reused with minimal changes (see Section 4).

How did the GUI improve usability?

The GUI removed the need to remember and type command syntax for actions like sending a private message. Online users are shown in a list and can be clicked directly, messages auto-scroll, and a status label makes the connection state visible at a glance - all of which reduce user error compared to a plain terminal interface.

Suggested future enhancement

A file-transfer feature could be added, allowing users to share files or images through the same TCP connection, encoded and chunked within the existing JSON message protocol.

9. Conclusion

The terminal-based chat application from Assignment 5 was successfully converted into a graphical desktop application using Tkinter, while reusing the server and core networking logic unchanged. The GUI client supports login validation, broadcast and private messaging, a live online-user list, and responsive background message handling. Testing across four Mininet clients and Wireshark packet capture confirmed correct end-to-end behaviour of the login, messaging, and disconnection flows.